

## Assignment 2: Policy Gradients

Due October 1, 11:59 pm

### 1 Introduction

The goal of this assignment is to experiment with policy gradient and its variants, including variance reduction tricks such as implementing reward-to-go and neural network baselines. The starter code can be found at

<https://github.com/LeCAR-Lab/16831-F26-HW/tree/main/hw2>

### 2 Review

#### 2.1 Policy gradient

Recall that the reinforcement learning objective is to learn a  $\theta^*$  that maximizes the objective function:

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}(\tau)} [r(\tau)] \quad (1)$$

where each rollout  $\tau$  is of length  $T$ , as follows:

$$\pi_{\theta}(\tau) = p(s_0, a_0, \dots, s_{T-1}, a_{T-1}) = p(s_0) \pi_{\theta}(a_0 | s_0) \prod_{t=1}^{T-1} p(s_t | s_{t-1}, a_{t-1}) \pi_{\theta}(a_t | s_t)$$

and

$$r(\tau) = r(s_0, a_0, \dots, s_{T-1}, a_{T-1}) = \sum_{t=0}^{T-1} r(s_t, a_t).$$

The policy gradient approach is to directly take the gradient of this objective:

$$\nabla_{\theta} J(\theta) = \nabla_{\theta} \int \pi_{\theta}(\tau) r(\tau) d\tau \quad (2)$$

$$= \int \pi_{\theta}(\tau) \nabla_{\theta} \log \pi_{\theta}(\tau) r(\tau) d\tau. \quad (3)$$

$$= \mathbb{E}_{\tau \sim \pi_{\theta}(\tau)} [\nabla_{\theta} \log \pi_{\theta}(\tau) r(\tau)] \quad (4)$$

$$(5)$$

In practice, the expectation over trajectories  $\tau$  can be approximated from a batch of  $N$  sampled trajectories:

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \nabla_{\theta} \log \pi_{\theta}(\tau_i) r(\tau_i) \quad (6)$$

$$= \frac{1}{N} \sum_{i=1}^N \left( \sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(a_{it} | s_{it}) \right) \left( \sum_{t=0}^{T-1} r(s_{it}, a_{it}) \right). \quad (7)$$

Here we see that the policy  $\pi_{\theta}$  is a probability distribution over the action space, conditioned on the state. In the agent-environment loop, the agent samples an action  $a_t$  from  $\pi_{\theta}(\cdot | s_t)$  and the environment responds with a reward  $r(s_t, a_t)$ .

#### 2.2 Variance Reduction

##### 2.2.1 Reward-to-go

One way to reduce the variance of the policy gradient is to exploit causality: the notion that the policy cannot affect rewards in the past. This yields the following modified objective, where the sum of rewards here does

not include the rewards achieved prior to the time step at which the policy is being queried. This sum of rewards is a sample estimate of the  $Q$  function, and is referred to as the “reward-to-go.”

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(a_{it}|s_{it}) \left( \sum_{t'=t}^{T-1} r(s_{it'}, a_{it'}) \right). \quad (8)$$

### 2.2.2 Discounting

Multiplying a discount factor  $\gamma$  to the rewards can be interpreted as encouraging the agent to focus more on the rewards that are closer in time, and less on the rewards that are further in the future. This can also be thought of as a means for reducing variance (because there is more variance possible when considering futures that are further into the future). We saw in lecture that the discount factor can be incorporated in two ways, as shown below.

The first way applies the discount on the rewards from full trajectory:

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \left( \sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(a_{it}|s_{it}) \right) \left( \sum_{t'=0}^{T-1} \gamma^{t'} r(s_{it'}, a_{it'}) \right) \quad (9)$$

and the second way applies the discount on the “reward-to-go:”

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(a_{it}|s_{it}) \left( \sum_{t'=t}^{T-1} \gamma^{t'-t} r(s_{it'}, a_{it'}) \right). \quad (10)$$

### 2.2.3 Baseline

Another variance reduction method is to subtract a baseline (that is a constant with respect to  $\tau$ ) from the sum of rewards:

$$\nabla_{\theta} J(\theta) = \nabla_{\theta} \mathbb{E}_{\tau \sim \pi_{\theta}(\tau)} [r(\tau) - b]. \quad (11)$$

This leaves the policy gradient unbiased because

$$\nabla_{\theta} \mathbb{E}_{\tau \sim \pi_{\theta}(\tau)} [b] = \mathbb{E}_{\tau \sim \pi_{\theta}(\tau)} [\nabla_{\theta} \log \pi_{\theta}(\tau) \cdot b] = 0.$$

In this assignment, we will implement a value function  $V_{\phi}^{\pi}$  which acts as a *state-dependent* baseline. This value function will be trained to approximate the sum of future rewards starting from a particular state:

$$V_{\phi}^{\pi}(s_t) \approx \sum_{t'=t}^{T-1} \mathbb{E}_{\pi_{\theta}} [r(s_{t'}, a_{t'}) | s_t], \quad (12)$$

so the approximate policy gradient now looks like this:

$$\nabla_{\theta} J(\theta) \approx \frac{1}{N} \sum_{i=1}^N \sum_{t=0}^{T-1} \nabla_{\theta} \log \pi_{\theta}(a_{it}|s_{it}) \left( \left( \sum_{t'=t}^{T-1} \gamma^{t'-t} r(s_{it'}, a_{it'}) \right) - V_{\phi}^{\pi}(s_{it}) \right). \quad (13)$$

### 2.2.4 Generalized Advantage Estimation

The quantity  $\left( \sum_{t'=t}^{T-1} \gamma^{t'-t} r(s_{t'}, a_{t'}) \right) - V_{\phi}^{\pi}(s_t)$  from the previous policy gradient expression (removing the  $i$  index for clarity) can be interpreted as an estimate of the advantage function:

$$A^{\pi}(s_t, a_t) = Q^{\pi}(s_t, a_t) - V^{\pi}(s_t), \quad (14)$$

where  $Q^{\pi}(s_t, a_t)$  is estimated using Monte Carlo returns and  $V^{\pi}(s_t)$  is estimated using the learned value function  $V_{\phi}^{\pi}$ . We can further reduce variance by also using  $V_{\phi}^{\pi}$  in place of the Monte Carlo returns to estimate the advantage function as:

$$A^{\pi}(s_t, a_t) \approx \delta_t = r(s_t, a_t) + \gamma V_{\phi}^{\pi}(s_{t+1}) - V_{\phi}^{\pi}(s_t), \quad (15)$$

with the edge case  $\delta_{T-1} = r(s_{T-1}, a_{T-1}) - V_\phi^\pi(s_{T-1})$ . However, this comes at the cost of introducing bias to our policy gradient estimate, due to modeling errors in  $V_\phi^\pi$ . We can instead use a combination of  $n$ -step Monte Carlo returns and  $V_\phi^\pi$  to estimate the advantage function as:

$$A_n^\pi(s_t, a_t) = \sum_{t'=t}^{t+n} \gamma^{t'-t} r(s_{t'}, a_{t'}) + \gamma^n V_\phi^\pi(s_{t+n+1}) - V_\phi^\pi(s_t). \quad (16)$$

Increasing  $n$  incorporates the Monte Carlo returns more heavily in the advantage estimate, which lowers bias and increases variance, while decreasing  $n$  does the opposite. Note that  $n = T - t - 1$  recovers the unbiased but higher variance Monte Carlo advantage estimate used in (13), while  $n = 0$  recovers the lower variance but higher bias advantage estimate  $\delta_t$ .

We can combine multiple  $n$ -step advantage estimates as an exponentially weighted sum, which is known as the generalized advantage estimator (GAE). Let  $\lambda \in [0, 1]$ . Then we define:

$$A_{GAE}^\pi(s_t, a_t) = \frac{1 - \lambda}{1 - \lambda^{T-t-1}} \sum_{n=1}^{T-t-1} \lambda^{n-1} A_n^\pi(s_t, a_t), \quad (17)$$

where  $\frac{1-\lambda}{1-\lambda^{T-t-1}}$  is a normalizing constant. Note that a higher  $\lambda$  emphasizes advantage estimates with higher values of  $n$ , and a lower  $\lambda$  does the opposite. Thus,  $\lambda$  serves as a control for the bias-variance tradeoff, where increasing  $\lambda$  decreases bias and increases variance. In the infinite horizon case ( $T = \infty$ ), we can show:

$$A_{GAE}^\pi(s_t, a_t) = (1 - \lambda) \sum_{n=1}^{\infty} \lambda^{n-1} A_n^\pi(s_t, a_t) \quad (18)$$

$$= \sum_{t'=t}^{\infty} (\gamma\lambda)^{t'-t} \delta_{t'}, \quad (19)$$

where we have omitted the derivation for brevity (see the GAE paper <https://arxiv.org/pdf/1506.02438.pdf> for details). In the finite horizon case, we can write:

$$A_{GAE}^\pi(s_t, a_t) = \sum_{t'=t}^{T-1} (\gamma\lambda)^{t'-t} \delta_{t'}, \quad (20)$$

which serves as a way we can efficiently implement the generalized advantage estimator, since we can recursively compute:

$$A_{GAE}^\pi(s_t, a_t) = \delta_t + \gamma\lambda A_{GAE}^\pi(s_{t+1}, a_{t+1}) \quad (21)$$

## 3 Overview of Implementation

### 3.1 Files

To implement policy gradients, we will be building up the code that we started in homework 1. All files needed to run your code are in the `hw2` folder, but there will be some blanks you will fill with your solutions from homework 1. These locations are marked with `# TODO: get this from hw1` and are found in the following files:

- `infrastructure/rl_trainer.py`
- `infrastructure/utils.py`
- `infrastructure/pytorch_util.py`
- `infrastructure/replay_buffer.py`
- `policies/MLP_policy.py`

After bringing in the required components from the previous homework, you can begin work on the new policy gradient code. These placeholders are marked with `# TODO`, located in the following files:

- `agents/pg_agent.py`
- `policies/MLP_policy.py`

The script to run the experiments is found in `scripts/run_hw2.py` (for the local option) or `scripts/run_hw2.ipynb` (for the Colab option).

### 3.2 Overview

As in the previous homework, the main training loop is implemented in `infrastructure/rl_trainer.py`.

The policy gradient algorithm uses the following 3 steps:

1. **Sample trajectories** by generating rollouts under your current policy.
2. **Estimate returns and compute advantages**. This is executed in the `train` function of `pg_agent.py`
3. **Train/Update parameters**. The computational graph for the policy and the baseline, as well as the update functions, are implemented in `policies/MLP_policy.py`.

## 4 Implementing Policy Gradients

You will be implementing two different return estimators within `pg_agent.py`. The first (“Case 1” within `calculate_q_vals`) uses the discounted cumulative return of the full trajectory and corresponds to the “vanilla” form of the policy gradient (Equation 9):

$$r(\tau_i) = \sum_{t'=0}^{T-1} \gamma^{t'} r(s_{it'}, a_{it'}). \quad (22)$$

The second (“Case 2”) uses the “reward-to-go” formulation from Equation 10:

$$r(\tau_i) = \sum_{t'=t}^{T-1} \gamma^{t'-t} r(s_{it'}, a_{it'}). \quad (23)$$

Note that these differ only by the starting point of the summation.

Implement these return estimators as well as the remaining sections marked `TODO` in the code. For the small-scale experiments, you may skip those sections that are run only if `nn_baseline` is `True`; we will return to baselines in Section 6. (These sections are in `MLPPolicyPG:update` and `PGAgent:estimate_advantage`.)

## 5 Small-Scale Experiments (7.5 Pts)

After you have implemented all non-baseline code from Section 4, you will run two small-scale experiments to get a feel for how different settings impact the performance of policy gradient methods.

**Experiment 1 (CartPole).** Run multiple experiments with the PG algorithm on the discrete `CartPole-v0` environment, using the following commands:

```
python rob831/scripts/run_hw2.py --env_name CartPole-v0 -n 150 -b 1500 \
    -dsa --exp_name q1_sb_no_rtg_dsa

python rob831/scripts/run_hw2.py --env_name CartPole-v0 -n 150 -b 1500 \
    -rtg -dsa --exp_name q1_sb_rtg_dsa

python rob831/scripts/run_hw2.py --env_name CartPole-v0 -n 150 -b 1500 \
    -rtg --exp_name q1_sb_rtg_na

python rob831/scripts/run_hw2.py --env_name CartPole-v0 -n 150 -b 6000 \
    -dsa --exp_name q1_lb_no_rtg_dsa

python rob831/scripts/run_hw2.py --env_name CartPole-v0 -n 150 -b 6000 \
    -rtg -dsa --exp_name q1_lb_rtg_dsa

python rob831/scripts/run_hw2.py --env_name CartPole-v0 -n 150 -b 6000 \
    -rtg --exp_name q1_lb_rtg_na
```

What's happening here:

- `-n` : Number of iterations.
- `-b` : Batch size (number of state-action pairs sampled while acting according to the current policy at each iteration).
- `-dsa` : Flag: if present, sets `standardize_advantages` to `False`. Otherwise, by default, standardizes advantages to have a mean of zero and standard deviation of one.
- `-rtg` : Flag: if present, sets `reward_to_go=True`. Otherwise, `reward_to_go=False` by default.
- `--exp_name` : Name for experiment, which goes into the name for the data logging directory.

Various other command line arguments will allow you to set batch size, learning rate, network architecture, and more. You can change these as well, but keep them fixed between the 6 experiments mentioned above.

### Deliverables for report:

- Create two graphs:
  - In the first graph, compare the learning curves (average return at each iteration) for the experiments prefixed with `q1_sb_`. (The small batch experiments.)
  - In the second graph, compare the learning curves for the experiments prefixed with `q1_lb_`. (The large batch experiments.)
- Answer the following questions briefly:
  - Which value estimator has better performance without advantage-standardization: the trajectory-centric one, or the one using reward-to-go?
  - Did advantage standardization help?
  - Did the batch size make an impact?

- Provide the exact command line configurations (or `#@params` settings in Colab) you used to run your experiments, including any parameters changed from their defaults.

**What to Expect:**

- The best configuration of CartPole in both the large and small batch cases should converge to a maximum score of 200.

**Experiment 2 (InvertedPendulum).** Run experiments on the `InvertedPendulum-v4` continuous control environment as follows:

```
python rob831/scripts/run_hw2.py --env_name InvertedPendulum-v4 \
  --ep_len 1000 --discount 0.92 -n 100 -l 2 -s 64 -b <b*> -lr <r*> -rtg \
  --exp_name q2_b<b*>_r<r*>
```

where your task is to find the smallest batch size `b*` and largest learning rate `r*` that gets to optimum (maximum score of 1000) in less than 100 iterations. The policy performance may fluctuate around 1000; this is fine. The precision of `b*` and `r*` need only be one significant digit.

**Deliverables:**

- Given the `b*` and `r*` you found, provide a learning curve where the policy gets to optimum (maximum score of 1000) in less than 100 iterations. (This may be for a single random seed, or averaged over multiple.)
- Provide the exact command line configurations you used to run your experiments.

## 6 Implementing Neural Network Baselines

You will now implement a value function as a state-dependent neural network baseline. This will require filling in some `TODO` sections skipped in Section 4. In particular:

- This neural network will be trained in the `update` method of `MLPPolicyPG` along with the policy gradient update.
- In `pg_agent.py:estimate_advantage`, the predictions of this network will be subtracted from the reward-to-go to yield an estimate of the advantage. This implements  $\left(\sum_{t'=t}^{T-1} \gamma^{t'-t} r(s_{it'}, a_{it'})\right) - V_{\phi}^{\pi}(s_{it})$ .

## 7 More Complex Experiments (4.5 Pts)

**Note:** The following tasks take quite a bit of time to train. Please start early! For all remaining experiments, use the reward-to-go estimator.

**Experiment 3 (LunarLander).** You will now use your policy gradient implementation to learn a controller for `LunarLanderContinuous-v2`. The purpose of this problem is to test and help you debug your baseline implementation from Section 6.

Run the following command:

```
python rob831/scripts/run_hw2.py \
  --env_name LunarLanderContinuous-v2 --ep_len 1000
  --discount 0.99 -n 100 -l 2 -s 64 -b 10000 -lr 0.005 \
  --reward_to_go --nn_baseline --exp_name q3_b10000_r0.005
```

**Deliverables:**

- Plot a learning curve for the above command. You should expect to achieve an average return of at least 120 by the end of training.

**Experiment 4 (HalfCheetah).** You will be using your policy gradient implementation to learn a controller for the HalfCheetah-v4 benchmark environment with an episode length of 150. This is shorter than the default episode length (1000), which speeds up training significantly. Search over batch sizes  $b \in [10000, 30000, 50000]$  and learning rates  $r \in [0.005, 0.01, 0.02]$  to replace  $\langle b \rangle$  and  $\langle r \rangle$  below.

```
python rob831/scripts/run_hw2.py --env_name HalfCheetah-v4 --ep_len 150 \
    --discount 0.95 -n 100 -l 2 -s 32 -b <b> -lr <r> -rtg --nn_baseline \
    --exp_name q4_search_b<b>_lr<r>_rtg_nnbaseline
```

#### Deliverables:

- Provide a single plot with the learning curves for the HalfCheetah experiments that you tried. Describe in words how the batch size and learning rate affected task performance. NOTE: for 7.2.2 simply plot from 7.2.1.

Once you've found optimal values  $b^*$  and  $r^*$ , use them to run the following commands (replace the terms in angle brackets):

```
python rob831/scripts/run_hw2.py --env_name HalfCheetah-v4 --ep_len 150 \
    --discount 0.95 -n 100 -l 2 -s 32 -b <b*> -lr <r*> \
    --exp_name q4_b<b*>_r<r*>

python rob831/scripts/run_hw2.py --env_name HalfCheetah-v4 --ep_len 150 \
    --discount 0.95 -n 100 -l 2 -s 32 -b <b*> -lr <r*> -rtg \
    --exp_name q4_b<b*>_r<r*>_rtg

python rob831/scripts/run_hw2.py --env_name HalfCheetah-v4 --ep_len 150 \
    --discount 0.95 -n 100 -l 2 -s 32 -b <b*> -lr <r*> --nn_baseline \
    --exp_name q4_b<b*>_r<r*>_nnbaseline

python rob831/scripts/run_hw2.py --env_name HalfCheetah-v4 --ep_len 150 \
    --discount 0.95 -n 100 -l 2 -s 32 -b <b*> -lr <r*> -rtg --nn_baseline \
    --exp_name q4_b<b*>_r<r*>_rtg_nnbaseline
```

**Deliverables:** Provide a single plot with the learning curves for these four runs. The run with both reward-to-go and the baseline should achieve an average score close to 200.

## 8 Implementing Generalized Advantage Estimation (3 Pts)

You will now use the value function you previously implemented to implement a simplified version of GAE- $\lambda$ . This will require filling in the remaining TODO section in `pg_agent.py:estimate_advantage`.

**Experiment 5 (Hopper).** You will now use your implementation of policy gradient with generalized advantage estimation to learn a controller for a version of Hopper-v4 with noisy actions. Search over  $\lambda \in [0, 0.95, 0.99, 1]$  to replace  $\langle \lambda \rangle$  below. Note that with a correct implementation,  $\lambda = 1$  is equivalent to the vanilla neural network baseline estimator. **Do not** change any of the other hyperparameters (e.g. batch size, learning rate).

```
python rob831/scripts/run_hw2.py \
    --env_name Hopper-v4 --ep_len 1000
    --discount 0.99 -n 300 -l 2 -s 32 -b 2000 -lr 0.001 \
    --reward_to_go --nn_baseline --action_noise_std 0.5 --gae_lambda <λ> \
    --exp_name q5_b2000_r0.001_lambda<λ>
```

#### Deliverables:

- Provide a single plot with the learning curves for the **Hopper-v4** experiments that you tried. Describe in words how  $\lambda$  affected task performance. The run with the best performance should achieve an average score close to 400.

**Open question:**

- Find one robotics paper from the last 5 years that uses policy gradients or an actor-critic method as part of its methodology. In a short paragraph (4–5 sentences): (i) cite the paper, (ii) briefly describe the robot task and how reinforcement learning is used, and (iii) identify one technique from this homework (e.g., reward-to-go, a value-function baseline, or GAE) that is related to the method used in the paper. A few sentences is enough; this is not meant to be an extensive literature review.

## 9 Submission

### 9.1 Submitting the PDF

Your report should be a PDF generated using the `hw2_submission.tex` template containing

- (a) All graphs and answers to short explanation questions requested for Experiments 1-5.
- (b) All command-line expressions you used to run your experiments.

### 9.2 Submitting the code and experiment runs

In order to turn in your code and experiment logs, create a folder that contains the following:

- A folder named `run_logs` with all the experiment runs from this assignment. These folders can be copied directly from the `rob831/data` folder. **Do not change the names originally assigned to the folders, as specified by `exp_name` in the instructions.** Video logging is disabled by default in the code, but if you turned it on for debugging, you need to run those again with `--video_log_freq -1`, or else the file size will be too large for submission.
- The `rob831` folder with all the `.py` files, with the same names and directory structure as the original homework repository (excluding the `rob831/data` folder). Also include any special instructions we need to run in order to produce each of your figures or tables in the form of a README file.

As an example, the unzipped version of your submission should result in the following file structure. **Make sure that the submit.zip file is below 15MB.**

If you are a Mac user, **do not use the default “Compress” option to create the zip.** It creates artifacts that the autograder does not like. You may use `zip -vr submit.zip submit -x "*.DS_Store"` from your terminal.

```
submit.zip
├── run_logs
│   ├── q1_lb_rtg_na.CartPole-v0.12-09-2019_17-53-4
│   │   └── events.out.tfevents.1567529456.e3a096ac8ff4
│   ├── q3_b40000_r0.005_LunarLanderContinuous-v2.12-09-2019_00-17-58
│   │   └── events.out.tfevents.1567529456.e3a096ac8ff4
│   └── ...
├── rob831
│   ├── agents
│   │   ├── pg_agent.py
│   │   └── ...
│   ├── policies
│   │   └── ...
│   └── ...
├── README.md
└── ...
```

### 9.3 Turning it in

Turn in your assignment by the deadline on Gradescope. Upload the zip file with your code to **HW2 Code**, and upload the PDF of your report to **HW2**.